

Algorithm Counter Method – C Programs

Clean GitHub-ready PDF. Code is stored as normal selectable text in a monospace font, without Markdown/code-fence syntax that can cause rendering problems.

1. Loop with Increasing Sum

Algorithm:

```
void function (int n)
{
    int i = 1;
    int s = 1;
    while (s <= n)
    {
        i++;
        s += i;
    }
}
```

Counter rule:

No counter increment for declarations, scanf(), and the final count-variable printf() statement.

C Program:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int count = 0;
    int i = 1;
    count++;
    int s = 1;
    count++;

    while (1)
    {
        count++;
        if (!(s <= n))
            break;

        i++;
        count++;
        s += i;
        count++;
    }

    printf("%d", count);
    return 0;
}
```

Sample:

Input: 9

Expected output: 12

Complexity:

The loop increases s by successively larger values, so the number of iterations grows on the order of $\sqrt{n}$.

2. Nested Loops with Immediate Break

Algorithm:

```
void func(int n)
{
    if (n == 1)
    {
        printf("*");
    }
    else
    {
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                printf("*");
                printf("*");
                break;
            }
        }
    }
}
```

C Program:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int count;
    if (n == 1)
    {
        count = 1;
    }
    else
    {
        count = 5 * n + 2;
    }

    printf("%d", count);
    return 0;
}
```

Sample:

Input: 2

Expected output: 12

Complexity:

Because the inner loop executes only once due to break, the running time is $O(n)$.

3. Factor Finder

Algorithm:

```
Factor(num)
{
    for (i = 1; i <= num; ++i)
    {
        if (num % i == 0)
        {
            printf("%d ", i);
        }
    }
}
```

C Program:

```
#include <stdio.h>

int main()
{
    int num;
    scanf("%d", &num);

    int count = 0;
    int i;

    for (i = 1; i <= num; i++)
    {
        count++;
        count++;

        if (num % i == 0)
        {
            count++;
        }
    }

    count++;
    printf("%d", count);
    return 0;
}
```

Samples:

Input: 12 -> Expected: 31

Input: 25 -> Expected: 54

Input: 4 -> Expected: 12

Complexity:

The loop runs num times, so the time complexity is $O(n)$.

4. Three Nested Loops with Powers of Two

Algorithm:

```
void function(int n)
{
    int c = 0;
    for (int i = n / 2; i < n; i++)
        for (int j = 1; j < n; j = 2 * j)
            for (int k = 1; k < n; k = k * 2)
                c++;
}
```

C Program:

```
#include <stdio.h>
#include <math.h>

int main()
{
    int n;
    scanf("%d", &n);

    int count = 1;

    for (int i = n / 2; i < n; i++)
    {
        count++;

        for (int j = 1; j < n; j = 2 * j)
        {
            count++;

            for (int k = 1; k < n; k = k * 2)
            {
                count++;
                count++;
            }
            count++;
        }
        count++;
    }

    count++;
    printf("%d", count);
    return 0;
}
```

Sample:

Input: 4

Expected output: 30

Complexity:

The outer loop executes about $n/2$ times. Each of the two inner loops doubles its variable, so each takes $O(\log n)$. T

5. Reverse an Integer

Algorithm:

```
void reverse(int n)
{
    int rev = 0, remainder;

    while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n /= 10;
    }

    print(rev);
}
```

C Program:

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int temp = n;
    int digits = 0;

    while (temp != 0)
    {
        digits++;
        temp /= 10;
    }

    int count = 4 * digits + 3;
    printf("%d", count);
    return 0;
}
```

Samples:

Input: 12 -> Expected: 11

Input: 1234 -> Expected: 19

Complexity:

If n has d digits, the loop executes d times. Therefore time complexity is $O(d)$, which is $O(\log n)$.